JAVA PROGRAMMING
MODULE - 2

INHERITANCE

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system). The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and felds of the parent class. Moreover, you can add new methods and felds in your current class also.

Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Why use inheritance in java?

For Method Overriding (so runtime polymorphism can be achieved). For Code Reusability.

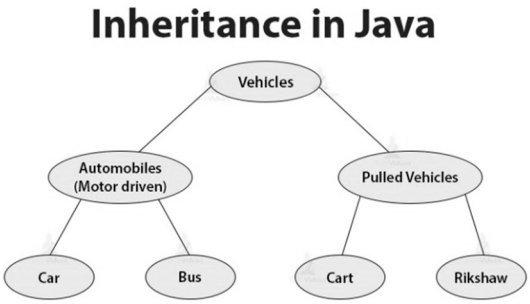
Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class**: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defned in the previous class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
        //methods and fields
}
```

The extends keyword indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality. In the terminology of Java, a class which is inherited is called a parent or superclass, and the new class is called child or subclass.




```
// Parent Class (Super Class)
class Animal
 {
        void eat()
        {
                System.out.println("This animal eats food.");
         }
}
// Child Class (Sub Class)
class Dog extends Animal
 {
        void bark()
        {
                System.out.println("Dog barks.");
        }
}
// Main Class
public class InheritanceExample
 {
         public static void main(String[] args)
        {
                Dog myDog = new Dog();
               myDog.eat();  // Inherited from Animal class
               myDog.bark(); // Defined in Dog class
        }
}
```


Java Inheritance (Subclass and Superclass)

In Java, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

- **subclass (child)** - the class that inherits from another class
- **superclass (parent)** - the class being inherited from

To inherit from a class, use the extends keyword.

```
package inheritance;
public class Parentclass
{
        void m1()
         {
                System.out.println("Superclass m1 method");
         }
 }
public class Childclass extends Parentclass
{
        void m2()
        {
                System.out.println("Childclass m2 method");
         }
 }
public class Test
 {
        public static void main(String[] args)
{
 // Creating an object of superclass.
   Parentclass p = new Parentclass();
   p.m1();
 // Here, subclass members are not available to superclass.
 // So, we cannot access them using superclass reference variable.
 }
}
```

Output:

Superclass m1 method

Types of Inheritance in Java

Below are the different types of inheritance which are supported by Java.

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

1. Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behaviour of a single-parent class. Sometimes, it is also known as simple inheritance. In

the below figure, ‘A’ is a parent class and ‘B’ is a child class. The class ‘B’ inherits all the properties of the class ‘A’.

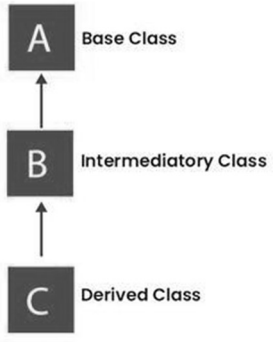
Single Inheritance

```
// Parent class
class Animal
{
        void eat()
       {
              System.out.println("This animal eats food.");
        }
}
// Child class inheriting from Animal
class Dog extends Animal
 {
        void bark()
       {
              System.out.println("Dog barks.");
       }
}
// Main class to create objects and call methods
public class Main
 {
       public static void main(String[] args)
       {
      // Creating an object of the Dog class
      Dog dog1 = new Dog();
      // Calling methods using the object
      dog1.eat();  // Inherited method from Animal class
      dog1.bark(); // Method from Dog class
        }
}
```

Output:
This animal eats food.
Dog barks.

2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the grandparent's members.



Multilevel Inheritance

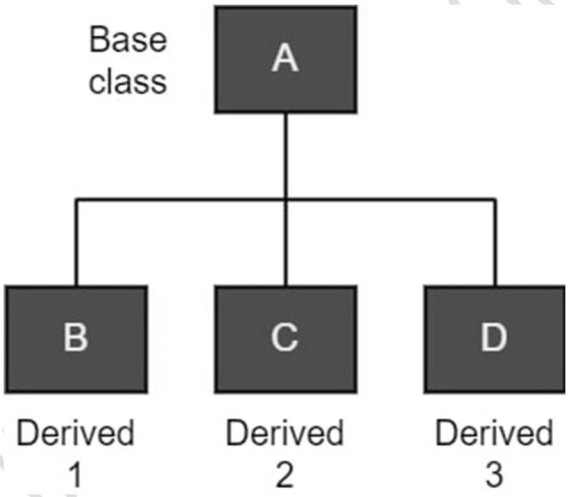
```
// Base Class
class Animal
{
   void eat()
  {
      System.out.println("Eating...");
   }
}
// Intermediate Class
class Mammal extends Animal
 {
   void walk()
  {
      System.out.println("Walking...");
   }
}
// Derived Class
class Human extends Mammal
{
  void talk()
  {
      System.out.println("Talking...");
   }
}
// Main Class
public class MultilevelInheritance
{
   public static void main(String[] args)
 {
      Human person = new Human();
```

```
        person.eat();  // From Animal class
        person.walk(); // From Mammal class
        person.talk(); // From Human class
    }
}
```

Output:
Eating...
Walking...
Talking...

3. Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.




```
// Parent Class
class Vehicle
 {
   void start()
   {
     System.out.println("Vehicle is starting...");
   }
}
// Child Class 1
class Car extends Vehicle
 {
       void drive()
{
      System.out.println("Car is driving...");
  }
```

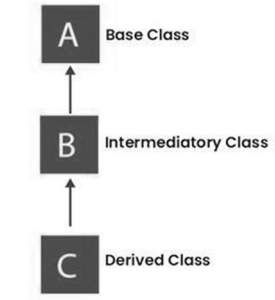
```
}
// Child Class 2
class Bike extends Vehicle
 {
   void ride()
  {
      System.out.println("Bike is riding...");
   }
}

// Main Class
public class HierarchicalInheritance
 {
   public static void main(String[] args)
  {
      Car myCar = new Car();
      myCar.start(); // Inherited from Vehicle class
      myCar.drive();
      Bike myBike = new Bike();
      myBike.start(); // Inherited from Vehicle class
      myBike.ride();
   }
}
```

Output:
Vehicle is starting...
Car is driving...
Vehicle is starting...
Bike is riding...

4. Multiple Inheritance (Through Interfaces)

In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. Please note that Java does not support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces. In the image below, Class C is derived from interfaces A and B.



Multilevel Inheritance

```
// First Interface
interface Animal
{
   void eat();
}
// Second Interface
interface Bird
{
   void fly();
}
// Implementing Multiple Interfaces
class Sparrow implements Animal, Bird
 {
   public void eat()
   {
      System.out.println("Sparrow eats grains.");
   }
   public void fly()
   {
      System.out.println("Sparrow flies in the sky.");
   }
}
// Main Class
public class MultipleInheritanceExample
 {
   public static void main(String[] args)
   {
      Sparrow mySparrow = new Sparrow();
      mySparrow.eat();
      mySparrow.fly();
   }
}
```

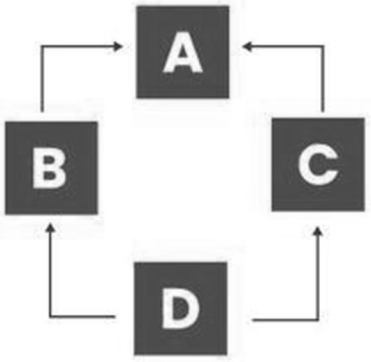
Output:
Sparrow eats grains.
Sparrow flies in the sky.

5. Hybrid Inheritance

It is a mix of two or more of the above types of inheritance. Since Java doesn't support multiple inheritances with classes, hybrid inheritance involving multiple inheritance is also not possible with classes. In Java, we can achieve hybrid inheritance only through Interfaces if we want to involve multiple inheritance to implement Hybrid inheritance.

However, it is important to note that Hybrid inheritance does not necessarily require the use of Multiple Inheritance exclusively. It can be achieved through a combination of Multilevel

Inheritance and Hierarchical Inheritance with classes, Hierarchical and Single Inheritance with classes. Therefore, it is indeed possible to implement Hybrid inheritance using classes alone, without relying on multiple inheritance type.



Hybrid Inheritance

```
// First Parent Interface
interface Parent1
 {
   void show();
}

// Second Parent Interface
interface Parent2
 {
   void display();
}
// Child Class implementing both interfaces
class Child implements Parent1, Parent2
 {
   public void show()
 {
      System.out.println("Parent1 method.");
   }

   public void display()
{
      System.out.println("Parent2 method.");
   }
}

// Main Class
public class HybridInheritanceExample
```

```
{
   public static void main(String[] args)
{
      Child obj = new Child();
      obj.show();
      obj.display();
   }
}
```

Output:
Parent1 method.
Parent2 method.

Overriding in Java

Overriding in Java occurs when a subclass (child class) implements a method that is already defined in the superclass or Base Class. The overriding method must have the same name, parameters, and return type as the method in the parent class. This allows subclasses to modify inherited behaviour while maintaining a consistent interface.

- The method in the subclass must have the same name, return type, and parameters as the method in the superclass.
- The method called is determined at runtime, allowing dynamic behaviour based on the object's type.
- Only non-static methods can be overridden.
- The@Override annotation ensures the method is correctly overridden and helps avoid errors.

Rules for Overriding:

- The method must have the same name as in the superclass.
- The method must have the same parameters as in the superclass.
- The method in the subclass should have an equal or more restrictive access modifier.
- The method cannot be static, final, or private in the superclass.

```
// Superclass
class Animal
{
   void sound()
    {
      System.out.println("Animals make sound.");
    }
```

```
}
// Subclass
class Dog extends Animal
{
   // Overriding the sound() method
   @Override
   void sound()
{
      System.out.println("Dog barks.");
   }
}
// Main Class
public class OverridingExample
{
   public static void main(String[] args)
{
      Animal myAnimal = new Animal();
      myAnimal.sound();  // Calls Animal's method
      Dog myDog = new Dog();
      myDog.sound();  // Calls Dog's overridden method
   }
}
```

Object Class in Java

Object class in Java is present in java.lang package. Every class in Java is directly or indirectly derived from the Object class. If a class does not extend any other class then it is a direct child class of the Java Object class and if it extends another class then it is indirectly derived. The Object class provides several methods such as toString(),equals(), hashCode(), and many others. Hence, the Object class acts as a root of the inheritance hierarchy in any Java Program.

```
// Java Code to demonstrate Object class
   class Person
   {
      String n;  //name
      // Constructor
      public Person(String n)
      {
         this.n = n;
      }
      // Override toString() for a
      // custom string representation
      @Override
```

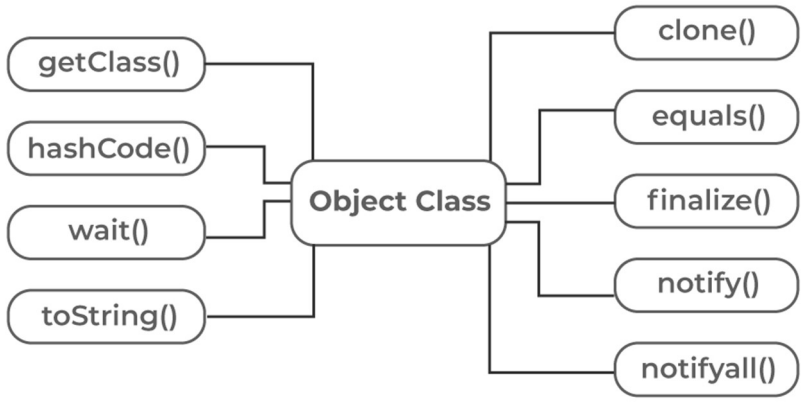
```
    public String toString()
   {
       return "Person{name:'" + n + "'}";
    }
    public static void main(String[] args)
    {
       Person p = new Person("Ram");
       // Custom string representation
       System.out.println(p.toString());
       // Default hash code value
       System.out.println(p.hashCode());
    }
}
```

Output:
Person{name:'Ram'}
321001045

Object Class Methods

The Object class provides multiple methods which are as follows:

- toString() method
- hashCode() method
- equals(Object obj) method
- finalize() method
- getClass() method
- clone() method



1. toString() Method

The toString() provides a String representation of an object and is used to convert an object to a String. The default toString() method for class Object returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object.

Example:

```
public class Student
{
   public String toString()
   {
            return "Student object";

   }

}
```

2. hashCode() Method

For every object, JVM generates a unique number which is a hashcode. It returns distinct integers for distinct objects. A common misconception about this method is that the hashCode() method returns the address of the object, which is not correct. It converts the internal address of the object to an integer by using an algorithm. The hashCode() method is native because in Java it is impossible to find the address of an object, so it uses native languages like C/C++ to find the address of the object.

Use of hashCode() method:

It returns a hash value that is used to search objects in a collection. JVM(Java Virtual Machine) uses the hashcode method while saving objects into hashing-related data structures like HashSet, HashMap, Hashtable, etc. The main advantage of saving objects based on hash code is that searching becomes easy.

Example:

```
public class Student
 {
   int roll;
  @Override
   public int hashCode()
    {
            return roll;

    }

}
```

3. equals(Object obj) Method

The equals() method compares the given object with the current object. It is recommended to override this method to define custom equality conditions.

Example:

```
public class Student
{
    int roll;
    @Override
     public boolean equals(Object o)
    {
                if (o instanceof Student)
                {
                        return this.roll == ((Student) o).roll;

                }

               return false;

      }

}
```

4. **getClass() method**

The getClass() method returns the class object of "this" object and is used to get the actual runtime class of the object. It can also be used to get metadata of this class. The returned Class object is the object that is locked by static synchronized methods of the represented class. As it is final so we don't override it.

Example:

```
public class Main
{
      public static void main(String[] args)
      {
         Object o = new String("Java");
         Class c = o.getClass();
         System.out.println("Class of Object o is: "+ c.getName());
      }
 }
```

5. finalize() method

The finalize() method is called just before an object is garbage collected. It is called the Garbage Collector on an object when the garbage collector determines that there are no more references to the object. We should override finalize() method to dispose of system resources,

perform clean-up activities and minimize memory leaks. For example, before destroying the Servlet objects web container, always called finalize method to perform clean-up activities of the session.

Example:

```
// Demonstrate working of finalize()
public class Program
{
   public static void main(String[] args)
 {

      Program t = new Program();
      System.out.println(t.hashCode());
      t = null;
      // calling garbage collector
      System.gc();
      System.out.println("end");
   }
   @Override protected void finalize()
   {
      System.out.println("finalize method called");
   }
}
```

6. clone() method

The clone() method creates and returns a new object that is a copy of the current object.

Example:

```
public class Book implements Cloneable
 {
   private String t;  //title
   public Book(String t)
   {
      this.t = t;
   }
   @Override
   public Object clone() throws CloneNotSupportedException
   {
      return super.clone();
   }
}
```

Polymorphism

Polymorphism in Java is a fundamental concept in object-oriented programming (OOP) that allows objects to behave differently based on their specific class type. The word polymorphism means having many forms. In Java, polymorphism refers to the ability of a message to be displayed in more than one form. This concept is a key feature of Object-Oriented Programming, and it allows objects to behave differently based on their specific class type.

Key features of polymorphism:

- Multiple Behaviors: One method can perform different actions based on the object.
- Method Overriding: A subclass provides its own implementation of a method defined in the parent class.
- Method Overloading: Multiple methods can have the same name but different parameters.
- Runtime Decision: The correct method is selected at runtime based on the actual object type.

Types of Polymorphism

In Java, polymorphism is broadly classified into two categories:

- **Compile-time polymorphism (static binding)**
- **Runtime polymorphism (dynamic binding)**

1. Compile-Time Polymorphism

Compile-time polymorphism is also known as static binding. This type of polymorphism can be achieved through function overloading or operator overloading. But in Java, it is restricted to function overloading as Java doesn't provide the support to cFunction Overloading

When there are at least two functions or methods with the same function name but either the number of parameters they contain is different or at least one data type of the corresponding parameter is different (or both), then it is known as a function or method overloading and these functions are known as overloaded functions.

Example :

```
class Main
{
    // Method 1
   // Method with 2 integer parameters
   static int Addition(int a, int b)
  {



    // Returns sum of integer numbers
    return a + b;
```

```
    }
    // Method 2
    // having the same name but with 2 double parameters
    static double Addition(double a, double b)
    {
        // Returns sum of double numbers
        return a + b;
    }
    public static void main(String args[])
  {
        // Calling method by passing
        // input as in arguments
        System.out.println(Addition(12, 14));
        System.out.println(Addition(15.2, 16.1));
    }

}
```

Runtime Polymorphism – Dynamic Binding

This is also known as dynamic binding. In this process, a function call made to a function is resolved during runtime only. We can achieve dynamic binding in Java through method overriding.

Method Overriding

Method overriding in Java occurs when a method in the base class has a definition in the derived class. The method or function in the base class is known as an overridden method.

```
// Class 1
class Parent
 {
    // Print method
    void Print()
    {
        // Print statement
        System.out.println("Inside Parent Class");
    }
}
// Class 2
class Child1 extends Parent
 {
    // Print method
    void Print()
    {
System.out.println("Inside Child1 Class");
}
```

```
        }
// Class 3
class Child2 extends Parent
{
   // Print method
   void Print()
   {
      // Print statement
      System.out.println("Inside Child2 Class");
   }
}
class Main
 {
   public static void main(String args[])
    {
      // Creating an object of class Parent
      Parent parent = new Parent();
      parent.Print();
      // Calling print methods
      parent = new Child1();
      parent.Print();
      parent = new Child2();
      parent.Print();
   }

}
```

Generic programming

- Generic programming allows you to write one code that works for different data types.
- Instead of writing multiple versions of a function or class for different types (like int, float, String), you can use generics to create a single flexible version.

Example:

```
class Box<T>
{
 // T is a placeholder for any type
    private T value;
   void setValue(T value)
 {
```

```
        this.value = value;
    }
    T getValue() {
        return value;
    }
}
public class Main {
    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();  // Works with Integer
        intBox.setValue(10);
        System.out.println(intBox.getValue()); // Output: 10
        Box<String> strBox = new Box<>();  // Works with String
        strBox.setValue("Hello");
        System.out.println(strBox.getValue()); // Output: Hello
    }
}
```

Class Type Casting in Java

In **Java polymorphism**, **casting** is used when we need to convert an object from one type to another, especially when dealing with **inheritance** and **interfaces**.

Types of Casting in Java

1. **Upcasting (Implicit Casting)**
2. **Downcasting (Explicit Casting)**

1. Upcasting is casting a subtype to a super type in an upward direction to the inheritance tree. It is an automatic procedure for which there are no efforts poured in to do so where a sub-class object is referred by a superclass reference variable. One can relate it with dynamic polymorphism.

```
class Animal
 {
```

```
    void makeSound()
   {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal
 {
    void bark()
 {
        System.out.println("Dog barks");
    }
}
public class Main
{
    public static void main(String[] args)
{
        Animal a = new Dog(); // Upcasting
        a.makeSound(); // Calls method from Animal class
      // a.bark(); // ERROR: Cannot call subclass-specific method
    }
}
```

2. Downcasting refers to the procedure when subclass type refers to the object of the parent class is known as downcasting. If it is performed directly compiler gives an error as ClassCastException is thrown at runtime. It is only achievable with the use of instanceof operator The object which is already upcast, that object only can be performed downcast

```
class Animal
 {
    void makeSound()
 {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal
{
    void bark()
 {
        System.out.println("Dog barks");
    }
}
```

```
public class Main
 {
   public static void main(String[] args)
 {
       Animal a = new Dog();
       // Upcasting (Implicit)
       Dog d = (Dog) a;
      // Downcasting (Explicit)
       d.bark();
      // Access subclass-specific method
    }
}
```

instanceof Operator

The instanceof operator in Java is used to check whether an object belongs to a specific class or subclass before performing operations like downcasting.
Syntax : object instanceof ClassName

Example:

```
class Animal
 {
   void makeSound()
 {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal
 {
   void bark()
{
      System.out.println("Dog barks");
   }
}
public class Main
 {
   public static void main(String[] args)
{
      Animal a = new Dog(); // Upcasting (Dog → Animal)

      if (a instanceof Dog) { // Checking before downcasting
```

```
            Dog d = (Dog) a; // Safe Downcasting
            d.bark(); // Calling subclass-specific method
        }
else
{
            System.out.println("Not a Dog instance, downcasting not possible!");
        }
    }
}
```

Abstract Class

In Java, abstract class is declared with the abstract keyword. It may have both abstract and non-abstract methods(methods with bodies). An abstract is a Java modifier applicable for classes and methods in Java but not for Variables. In this article, we will learn the use of abstract classes in Java.

What is Abstract Class in Java?
Java abstract class is a class that can not be instantiated by itself, it needs to be subclassed by another class to use its properties. An abstract class is declared using the "abstract" keyword in its class definition.

Example:

```
classabstract class Shape
{
   int color;
   // An abstract function
   abstract void draw();
}
```

In Java, the following some important observations about abstract classes are as follows:

- An instance of an abstract class can not be created.
- Constructors are allowed.
- We can have an abstract class without any abstract method.
- There can be a final method in abstract class but any abstract method in class(abstract class) can not be declared as final or in simpler terms final method can not be abstract itself as it will yield an error: "Illegal combination of modifiers: abstract and final"
- We can define static methods in an abstract class
- We can use the abstract keyword for declaring top-level classes (Outer class) as well as inner classes as abstract

If a class contains at least one abstract method then compulsory should declare a class as abstract
If the Child class is unable to provide implementation to all abstract methods of the Parent class then we should declare that Child class as abstract so that the next level Child class should provide implementation to the remaining abstract methodExamples of Java Abstract Class

Below is the implementation of the above topic:

```
// Abstract class
abstract class Sunstar
 {
   abstract void printInfo();
}
// Abstraction performed using extends
class Employee extends Sunstar
 {
   void printInfo()
   {
      String name = "avinash";
      int age = 21;
      float salary = 222.2F;
      System.out.println(name);
      System.out.println(age);
      System.out.println(salary);
   }
}
// Base class
class Base
 {
   public static void main(String args[])
   {
      Sunstar s = new Employee();
      s.printInfo();
   }
}
```

Output
avinash
21
222.2

INTERFACES

Difference between Abstract Class and Interfaces

Abstract Class	Interfaces
Cannot be instantiated; contains both abstract (without implementation) and concrete methods (with implementation)	Specifies a set of methods a class must implement; methods are abstract by default.
Can have both implemented and abstract methods.	Methods are abstract by default; Java 8, can have default and static methods.
class can inherit from only one abstract class.	A class can implement multiple interfaces.
Methods and properties can have any access modifier (public, protected, private).	Methods and properties are implicitly public.
Can have member variables (final, non-final, static, non-static).	Variables are implicitly public, static, and final (constants).
Can extend another abstract class or a concrete class	Can extend multiple interfaces
Can have final, non-final, static, and non-static variables	Only has public, static, and final variables
Used when classes share common behavior	Used to define a contract for implementing classes

Defining an Interface

- An **interface** in Java is a blueprint of a class that contains **only abstract methods** and **static or default methods**. Interfaces help achieve **abstraction** and **multiple inheritance** in Java.
- Syntax for Defining an Interface

```
interface InterfaceName {
    // Abstract method (method without a body)
    void method1();
    void method2();
}
```

1.Implementing an Interface

- A class implements an interface using the implements keyword and must provide implementations for all its abstract methods.

```
interface Animal {
    void makeSound(); // Abstract method
}
// Implementing the interface in a class
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Dog barks");
    }
}
public class InterfaceExample {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.makeSound(); // Output: Dog barks
    }
     }
```

2.Extending an Interface

- An interface can extend another interface using the extends keyword. This allows an interface to inherit methods from another interface.

```
interface Animal {
    void eat();
}
// Child interface extending the parent interface
interface Dog extends Animal {
    void bark();
}
// Implementing the extended interface in a class
```

```
class Labrador implements Dog
{
    public void eat() {
        System.out.println("Labrador is eating.");
    }
    public void bark() {
        System.out.println("Labrador barks loudly.");
    }
}
public class InterfaceExtendingExample {
    public static void main(String[] args) {
        Labrador myLab = new Labrador();
        myLab.eat();  // Output: Labrador is eating.
        myLab.bark(); // Output: Labrador barks loudly.
    }
}
```

PACKAGES

- Packages in Java are a mechanism that encapsulates a group of classes, sub-packages, and interfaces. Packages are used for:
- Prevent naming conflicts by allowing classes with the same name to exist in different packages, like college.staff.cse.Employee and college.staff.ee.Employee.
- They make it easier to organize, locate, and use classes, interfaces, and other components.
- Packages also provide controlled access for Protected members that are accessible within the same package and by subclasses.
- Also for default members (no access specifier) that are accessible only within the same package.

By grouping related classes into packages, Java promotes data encapsulation, making code reusable and easier to manage. Simply import the desired class from a package to use it in your program.

Example:

import java.util.*;

Here, util is a sub-package created inside the java package.

Accessing Classes Inside a Package

In Java, we can import classes from a package using either of the following methods:

1. Import a specific class:

import java.util.Vector;

This imports only the Vector class from the java.util package.

2. Import all classes from a package:

import java.util.*;

Example:

Importing Class

```
// Import the Vector class from
// the java.util package
import java.util.Vector;
public class Java {
    public Java() {
        // java.util.Vector is imported, hence we are
        // able to access it directly in our code.
        Vector v = new Vector();
        // java.util.ArrayList is not imported, hence
        // we are referring to it using the complete
        // package name.
        java.util.ArrayList l = new java.util.ArrayList();
    }

    public static void main(String[] args)
```

```
{
    // class to invoke the constructor
    new Java();
  }
}
```

Types of Java Packages

- Built-in Packages
- User-defined Packages

1. Built-in Packages

- These packages consist of a large number of classes which are a part of Java API.Some of the commonly used built-in packages are:
- java.lang: Contains language support classes(e.g classes which defines primitive data types, math operations). This package is automatically imported.
- java.io: Contains classes for supporting input / output operations.
- java.util: Contains utility classes which implement data structures like Linked List, Dictionary and support ; for Date / Time operations.
- java.applet: Contains classes for creating Applets.
- java.awt: Contain classes for implementing the components for graphical user interfaces (like button , ;menus etc)
- java.net: Contain classes for supporting networking operations.

2. User-defined Packages

These are the packages that are defined by the user.

1. Create the Package:

First we create a directory myPackage (name should be same as the name of the package). Then create the MyClass inside the directory with the first statement being the package names.

```
// Name of the package must be same as the directory
// under which this file is saved
package myPackage;


public class MyClass
{
```

```
    public void getNames(String s)
    {
        System.out.println(s);
    }
}
```

2. Use the Class in Program:

Now we will use the MyClass class in our program.

```
// import 'MyClass' class from 'names' myPackage
import myPackage.MyClass;
public class Java {
  public static void main(String args[]) {
    // Initializing the String variable
    // with a value
    String s = "JAVA PROGRAMMING";
    // Creating an instance of class MyClass in
    // the package.
    MyClass o = new MyClass();
    o.getNames(s);
  }
}
```

Understanding CLASSPATH

Classpath in Java is the path where the Java Virtual Machine (JVM) and Java compiler look for .class files when executing a program. It defines where Java should search for compiled class files, packages, or external libraries**.**

- By default, Java looks for classes in the current directory (.) unless specified otherwise.

Setting Classpath in Java

- Default Classpath (Current Directory .)

- If no classpath is specified, Java searches for classes in the current directory.

Example: Running a Simple Java Program Without Setting Classpath

```
// File: HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java Classpath!");
    }
}
```

- Compile & Run Without Classpath

```
javac HelloWorld.java   # Compiling (stores HelloWorld.class in the same directory)
java HelloWorld         # Running (JVM finds HelloWorld.class in the current directory)
```

Importing Packages

- A package in Java is a way to organize related classes and interfaces into a structured directory. Importing a package allows us to use its classes without specifying the full package name every time.
- **Types of Imports in Java**
  1. Importing a Specific Class
- We can import a single class from a package using the import keyword.
- Example: Importing a Single Class

```
import java.util.Scanner;  // Importing Scanner class
public class SingleImportExample
 {
   public static void main(String[] args)
  {
       Scanner input = new Scanner(System.in);
       System.out.print("Enter your name: ");
       String name = input.nextLine();
```

```
        System.out.println("Hello, " + name);
        input.close();
    }
}
```

Importing All Classes in a Package (* Import)

- The import static keyword allows importing static methods or variables directly.

Example: Using Math Class Without Math. Prefix

```
import static java.lang.Math.*;  // Import all static methods of Math class
public class StaticImportExample
{
    public static void main(String[] args)
 {
        System.out.println("Square root of 25: " + sqrt(25));  // No need for Math.sqrt()
        System.out.println("PI value: " + PI);  // No need for Math.PI
    }
}
```

Importing Static Members (static import)

- The import static keyword allows importing static methods or variables directly.

Example: Using Math Class Without Math. Prefix

```
import static java.lang.Math.*;  // Import all static methods of Math class
public class StaticImportExample
{
    public static void main(String[] args)
 {
        System.out.println("Square root of 25: " + sqrt(25));  // No need for Math.sqrt()
        System.out.println("PI value: " + PI);  // No need for Math.PI
    }
```

}